PROJECT SYNOPSIS

AUTOSAR HLD Document Analysis Assistant

An Evidence-Grounded AI System for Automotive Software Architecture Analysis

Submitted as part of: TechPulse FY-26 Applied AI/ML Capstone Project

Submitted by: Saurabh Mali
Institution: Pimpri Chinchwad College of Engineering (PCCOE), Pune
Department: Computer Engineering
Academic Year: 2026–2027

---

1. Abstract

The automotive industry relies on detailed High-Level Design (HLD) documents to define software components, interfaces, ports, signals, dependencies, and functional interactions within complex automotive software systems. These documents often contain large amounts of technical information in PDF format, making manual analysis time-consuming and difficult to maintain consistently. Engineers must frequently search across sections, verify component responsibilities, trace interfaces, identify inconsistencies, and compare design revisions.

The **AUTOSAR HLD Document Analysis Assistant** is an AI-powered application designed to simplify these activities through document processing, semantic search, Retrieval-Augmented Generation (RAG), and evidence-grounded question answering. The system extracts text and relevant metadata from supported documents, divides the extracted content into context-aware chunks, converts these chunks into numerical embeddings, and stores them in a local ChromaDB vector database. When a user submits a question, the application retrieves relevant evidence and uses a configured Gemini language model to generate a response supported by source citations.

The application provides a Streamlit-based interface with modules for document management, architecture-related question answering, architecture exploration, issue and inconsistency analysis, revision comparison, and result export. Source citations

containing document names, page numbers, sections, and evidence passages help engineers verify generated answers against the original documentation.

The prototype was evaluated using a synthetic AUTOSAR HLD document and a benchmark of 20 questions. The reported evaluation achieved 100% Recall@4, 100% citation validity, 100% citation coverage, 100% abstention accuracy, an 89.1% factual match rate, an average query latency of 1.403 seconds, and document ingestion time of 0.60 seconds. These results demonstrate promising retrieval and citation performance on the test dataset, while broader validation on real industrial documents remains necessary.

Keywords: AUTOSAR, High-Level Design, Artificial Intelligence, Retrieval-Augmented Generation, Large Language Models, ChromaDB, Semantic Search, Gemini API, Software Architecture, Explainable AI.

---

2. Introduction

Modern automotive software systems contain interconnected electronic control units, software components, communication interfaces, and functional dependencies. The AUTOSAR architecture provides standardized approaches for organizing automotive software components and their interactions.

High-Level Design documents describe the architecture and behavior of these systems. They may include component definitions, provided and required ports, interfaces, signal information, dependencies, functional flows, and integration details. Engineers use these documents during architecture design, software development, integration, testing, and maintenance.

However, the information is frequently distributed across lengthy documents containing paragraphs, tables, diagrams, and technical terminology. Finding a specific component's responsibility or identifying the interface through which it communicates can require repeated manual searches. Comparing document revisions introduces further complexity because a small change in a port definition or dependency may affect several related components.

Traditional keyword-based search is useful for finding exact terms but may fail to identify relevant information when a question uses different wording from the original document. General-purpose language models can interpret natural language questions,

but without appropriate grounding, they may generate unsupported or inaccurate technical information.

The proposed AUTOSAR HLD Document Analysis Assistant addresses these challenges by combining semantic document retrieval with a language model that generates answers using retrieved evidence. Instead of treating the model as the sole source of information, the application connects each answer to the relevant document passages so that users can inspect and verify the results.

The system is intended to assist engineering teams rather than replace qualified engineering judgment or formal architecture approval processes.

3. Problem Statement

Automotive software architecture information is often maintained in complex HLD documents that require substantial manual effort to search, interpret, validate, and compare. Engineers may encounter difficulties locating component definitions, identifying provided and required ports, tracing dependencies, verifying interface specifications, and detecting inconsistencies between revisions.

Conventional keyword search does not always retrieve semantically relevant passages, while an ungrounded language model may generate answers that are not supported by the source document. Furthermore, the lack of direct references to document pages and sections makes independent verification difficult.

Therefore, there is a need for an AI-assisted document analysis system that can extract and organize architectural information, retrieve relevant evidence using natural language queries, generate source-grounded responses, support consistency analysis and revision comparison, and provide traceable outputs for human review.

4. Motivation

The motivation for this project is to improve the accessibility, traceability, and reviewability of automotive software architecture documentation.

The major motivations are:

1. **Reducing manual review effort:** Minimize repeated searches through lengthy HLD documents.

2. **Improving information accessibility:** Allow engineers to ask architecture-related questions in natural language.
3. **Supporting traceability:** Connect generated answers to document names, pages, sections, and evidence passages.
4. **Improving consistency analysis:** Assist in identifying potential mismatches, missing information, and conflicting definitions.
5. **Supporting engineering collaboration:** Make architecture knowledge easier to explore and communicate across development and testing teams.
6. **Encouraging verifiable AI:** Use retrieved document evidence rather than relying solely on the language model's general knowledge.
7. **Supporting maintainability:** Help engineers examine changes between document revisions and understand their potential implications.

5. Aim and Objectives

5.1 Aim

To design and implement an AI-assisted application that automates the extraction, organization, semantic retrieval, and evidence-grounded analysis of AUTOSAR High-Level Design documents.

5.2 Specific objectives

- Develop a document ingestion pipeline for supported HLD PDF files.
- Extract document text, headings, page information, and relevant structural metadata.
- Divide extracted content into context-aware chunks suitable for semantic retrieval.
- Generate vector embeddings using the all-MiniLM-L6-v2 embedding model.
- Store document chunks and associated metadata in ChromaDB.
- Enable natural language queries through a Streamlit-based interface.
- Integrate the Gemini API to generate responses grounded in retrieved evidence.
- Display citations that help users verify answers against source documents.
- Support architecture exploration and component-level information retrieval.
- Provide functionality for potential issue detection, inconsistency analysis, and document revision comparison.
- Allow findings and analysis results to be exported for further review.
- Evaluate retrieval quality, citation quality, factual agreement, abstention behavior, and processing time.

6. Scope of the Project

6.1 In-scope functionalities

Document processing

- Uploading and selecting supported architecture documents.
- Extracting textual content and document metadata.
- Processing supported scanned pages through OCR when configured.
- Organizing extracted content for downstream retrieval.

Architecture information retrieval

- Searching for component responsibilities.
- Identifying provided and required ports.
- Retrieving interface and dependency information.
- Answering natural language questions about the document.
- Presenting source references and retrieved evidence.

AI-assisted analysis

- Generating answers from retrieved passages using Gemini.
- Identifying potential issues and inconsistencies.
- Supporting architecture exploration and revision comparison.
- Exporting available analysis findings.

Evaluation and reporting

- Executing a benchmark of predefined questions.
- Measuring retrieval and citation performance.
- Measuring factual match rate and response latency.
- Recording evaluation results and known limitations.

6.2 Out-of-scope functionalities

- Automatically approving or rejecting architecture decisions.
- Modifying original HLD documents without human review.
- Replacing formal architecture governance and engineering sign-off.
- Certifying compliance with AUTOSAR requirements or ISO 26262.
- Guaranteeing correctness across every possible automotive architecture document.

- Establishing production readiness solely from synthetic benchmark results.

The system provides analytical assistance; final engineering decisions remain the responsibility of qualified professionals.

7. Proposed System

The proposed solution is a Python-based web application that combines semantic search, a local vector database, and a large language model.

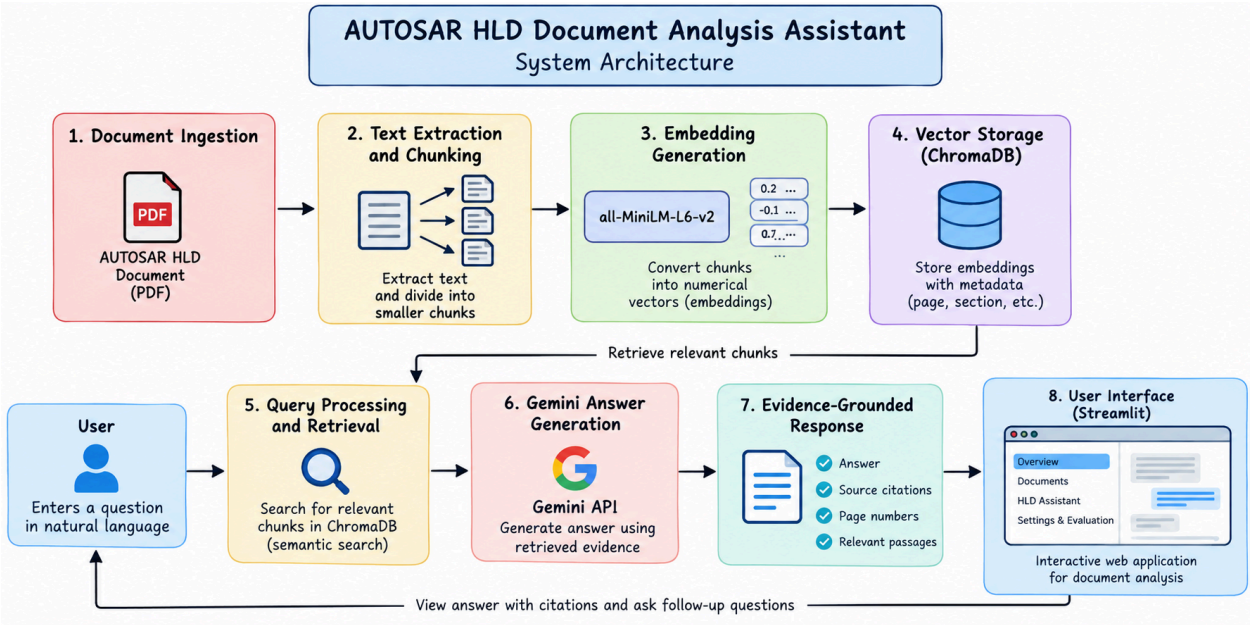
The system follows a Retrieval-Augmented Generation workflow. Document content is processed and indexed before users submit questions. For each query, the application retrieves relevant document passages and supplies them as evidence to the language model. The resulting answer is displayed with citations so that users can verify the claims.

The major modules are:

1. **Document Management Module:** Manages available HLD documents and their ingestion.
2. **Text Extraction and Chunking Module:** Extracts document content and divides it into retrievable units.
3. **Embedding Module:** Converts text chunks into vector representations.
4. **Vector Storage and Retrieval Module:** Stores and searches document embeddings using ChromaDB.
5. **Gemini Integration Module:** Generates answers from the retrieved context.
6. **Evidence and Citation Module:** Associates answers with relevant document passages and source metadata.
7. **Architecture Analysis Module:** Supports architecture exploration, issue identification, and comparison workflows.
8. **Evaluation Module:** Calculates and records benchmark metrics.
9. **Export Module:** Helps users preserve and share analysis results.

8. System Architecture

The application is organized into presentation, processing, retrieval, AI generation, and evaluation layers.



8.1 Architectural workflow

Input document → Text extraction → Context-aware chunking → Embedding generation → ChromaDB indexing → Semantic retrieval → Gemini answer generation → Citation verification → User-facing results

The workflow can be divided into two phases.

Phase A: Document indexing

1. The user uploads or selects an HLD document.
2. The ingestion pipeline extracts available text and metadata.
3. Extracted content is divided into manageable chunks while preserving relevant context.
4. The embedding model generates a vector representation for each chunk.
5. The chunks, embeddings, and metadata are stored in ChromaDB.

Phase B: Query processing

1. The user enters an architecture-related question.
2. The application converts the query into an embedding.
3. The retrieval component searches for semantically relevant chunks.
4. The selected evidence is supplied to the configured Gemini model.
5. The model generates an answer using the retrieved context.

6. The application displays the response with document citations and supporting passages.
7. The user reviews the evidence and can continue with other analysis or export operations.

If the Gemini API is unavailable, retrieval-only behavior can provide evidence-based results without claiming that a language-model-generated answer was produced.

8.2 Major architectural components

Component	Responsibility
Streamlit	User interface and navigation
Python	Application logic and processing
PDF processing utilities	Text and page extraction
all-MiniLM-L6-v2	Text embedding generation
ChromaDB	Persistent vector storage and semantic retrieval
Gemini API	Context-grounded answer generation
Evaluation scripts	Benchmark execution and metric calculation
Local evaluation artifacts	Storage of benchmark results and reports

9. Methodology

The project follows a modular implementation and evaluation methodology.

9.1 Requirement analysis

The first phase identifies common difficulties in HLD analysis, including lengthy documents, dispersed architecture information, manual search, revision differences, and limited traceability.

The requirements are translated into application capabilities such as document ingestion, semantic retrieval, cited question answering, issue analysis, comparison, and export.

9.2 Document ingestion

The document ingestion component accepts supported PDF documents and extracts their available textual content. Page numbers and section-related metadata are retained where available to support traceability.

Scanned documents may require OCR to extract text from page images. OCR is a text-extraction mechanism and does not, by itself, guarantee correct interpretation of architectural diagrams, connector arrows, or visual relationships.

9.3 Context-aware chunking

Extracted text is divided into smaller sections called chunks. Chunking allows the retrieval system to identify relevant portions of a document without supplying the entire document for every query.

Where possible, chunk metadata retains useful information such as document name, page number, and section. This improves the ability to display evidence and trace answers to their sources.

9.4 Embedding generation

The project uses all-MiniLM-L6-v2 to convert text chunks into numerical vector representations.

Semantically related passages can have similar vector representations even when they do not contain exactly the same keywords. This enables semantic retrieval for natural language questions.

9.5 Vector indexing and retrieval

ChromaDB stores the embedded chunks together with associated metadata. When a query is submitted, the retrieval component searches the vector store and selects the most relevant passages.

The application can then inspect the retrieved evidence and present it to the user. Retrieval quality is evaluated using Recall@4 on the project's benchmark dataset.

9.6 Retrieval-Augmented Generation

The application integrates a configured Gemini language model with the retrieval pipeline.

The retrieved passages provide the contextual evidence for answer generation. The model is instructed to answer using the supplied evidence, while the application presents document references to support verification.

This architecture separates document retrieval from language generation. A correct answer depends on both retrieving suitable evidence and generating a faithful response from that evidence.

9.7 Citation and evidence validation

The application displays available source information, including document name, page number, section, and retrieved evidence passages.

Citation evaluation checks whether generated references are valid and whether answers contain the expected source coverage under the project's evaluation rules. Users can inspect retrieved passages instead of accepting an answer without verification.

9.8 Architecture analysis and comparison

The application includes dedicated areas for architecture exploration, potential issues and inconsistencies, and document revision comparison.

These functions are intended to help engineers investigate component definitions and potential differences. Their outputs should be treated as candidate findings requiring verification against the relevant source documents.

9.9 Evaluation

The system is tested using a synthetic AUTOSAR HLD document and a predefined set of 20 questions. The evaluation measures retrieval recall, citation validity, citation coverage, abstention accuracy, factual match rate, query latency, and document ingestion time.

Results are stored in evaluation artifacts and summarized in a report. The results are interpreted in the context of the test dataset rather than as a guarantee of performance on all real-world HLD documents.

10. Implementation Details

10.1 User interface

The application uses Streamlit to provide a browser-based interface. Its navigation includes:

- **Overview:** Presents a summary of the system and its status.
- **Documents:** Supports document selection and ingestion-related operations.
- **HLD Assistant:** Accepts natural language questions and displays evidence-grounded answers.
- **Settings & Evaluation:** Provides configuration-related and evaluation functionality.
- **Architecture Explorer:** Supports inspection of architectural information.
- **Issues & Inconsistencies:** Supports investigation of potential architecture problems.
- **Compare Revisions:** Supports comparison of document versions.
- **Export:** Provides access to supported result-export operations.

10.2 Vector database

ChromaDB is used for persistent storage and semantic retrieval of document chunks. The vector store is maintained separately from the source PDF, allowing the application to reuse indexed document content across queries.

The indexing process must preserve consistency between the source document, chunk metadata, and stored embeddings. If an index is cleared, the document must be reindexed before normal retrieval can resume.

10.3 Language-model integration

The Gemini API provides language-model generation for queries where the integration is configured and available.

The application distinguishes API configuration and connection status from vector-store readiness. This is important because a functioning vector database does not necessarily imply that language-model generation is available.

The model's output is presented with source evidence, and the system should preserve retrieval-only behavior when Gemini is unavailable.

10.4 Evaluation artifacts

The implementation includes evaluation scripts and stored results for documenting benchmark execution. These artifacts support repeatability, metric inspection, and technical reporting.

The reported evaluation uses a synthetic PDF with six pages and six indexed chunks. This is a small controlled dataset and should not be interpreted as a representative sample of industrial-scale HLD documentation.

11. Experimental Setup

The reported benchmark configuration is summarized below.

Parameter	**Configuration**
Application	AUTOSAR HLD Document Analysis Assistant
Evaluation dataset	Synthetic AUTOSAR HLD document
Document	synthetic_autosar_hld.pdf
Document length	6 pages
Indexed chunks	6
Number of evaluation questions	20
Embedding model	all-MiniLM-L6-v2
Vector database	ChromaDB
Language-model integration	Gemini API
Evaluation mode	Offline retrieval evaluation
Reported evaluation timestamp	11 October 2026, 06:50:53 UTC

The reported benchmark is an offline retrieval evaluation. Its metrics should not be described as a complete measurement of live Gemini-generated answer quality unless the evaluation pipeline explicitly performs and evaluates those live generations.

12. Evaluation Metrics

12.1 Recall@4

Recall@4 measures whether the retrieval system finds the relevant evidence within the first four retrieved results, according to the benchmark's relevance annotations.

$$Recall@4 = \frac{Relevantitemsretrievedintop4}{Totalrelevantitems} \times 100$$

The reported value is 100%.

12.2 Citation Validity

Citation validity measures the proportion of evaluated citations that satisfy the project's citation-validation rules.

$$CitationValidity = \frac{Validcitations}{Totalevaluatedcitations} \times 100$$

The reported value is 100%.

12.3 Citation coverage

Citation coverage measures how well the evaluated answers provide the expected source references under the benchmark's coverage rules.

The reported value is 100%.

12.4 Abstention accuracy

Abstention accuracy measures performance on evaluation cases that require the system to refrain from providing an unsupported answer, according to the test labels.

The reported value is 100%. This result applies to the tested questions and does not establish perfect abstention behavior for arbitrary user queries.

12.5 Factual match rate

Factual match rate measures agreement between evaluated answers and the expected factual content according to the benchmark's scoring procedure.

The reported value is 89.1%.

12.6 Query latency

Query latency measures the time taken to process a query under the evaluation setup.

The reported average is 1.403 seconds, with a target of less than 2 seconds.

12.7 Document ingestion time

Document ingestion time measures the time required to process and index the test document under the recorded evaluation conditions.

The reported value is 0.60 seconds, with a target of less than 10 seconds.

13. Results and Discussion

The reported results are summarized below.

Evaluation parameter	**Reported result**
Recall@4	100%
Citation validity	100%
Citation coverage	100%
Abstention accuracy	100%
Factual match rate	89.1%
Average query latency	1.403 seconds

Document ingestion time	0.60 seconds

The benchmark indicates that the system successfully retrieved the expected evidence for the tested questions and met the stated latency targets under the recorded conditions. The citation metrics also indicate strong source traceability for the evaluated dataset.

The factual match rate of 89.1% shows that retrieval and citation success should not be treated as equivalent to perfect answer correctness. The remaining gap motivates additional tests for ambiguous questions, incomplete documentation, contradictory definitions, and cases where retrieved evidence is insufficient.

The evaluation dataset is small and synthetic. Consequently, the reported metrics establish prototype performance on the benchmark, not guaranteed performance on large, complex, or safety-critical automotive architecture documents.

The evaluation results should be reproduced using the saved benchmark artifacts before being used as formal evidence in a production-readiness claim.

14. Expected Outcomes and Benefits

The project is designed to provide the following benefits:

1. **Faster information retrieval:** Engineers can search architectural knowledge using natural language rather than manually navigating every document section.
2. **Improved traceability:** Answers can include page, section, and evidence references.
3. **More accessible documentation:** Component responsibilities and interface details become easier to investigate.
4. **Support for consistency review:** Potential mismatches and missing information can be highlighted for further examination.
5. **Revision analysis support:** Engineers can investigate differences between document versions.
6. **Reusable architecture knowledge:** Indexed document content can be retrieved repeatedly without reprocessing the entire document for each question.
7. **Improved review workflow:** Structured findings and exports can support technical discussions and documentation reviews.

8. **Human-verifiable AI output:** Engineers can inspect the cited evidence before relying on an answer.

These benefits are intended outcomes of the system. Their actual magnitude in an industrial setting would need to be measured through user studies and evaluation on representative documents.

15. Limitations

The current prototype has several limitations:

- The reported benchmark uses a small synthetic document rather than a large collection of real industrial HLDs.
- The factual match rate is below 100%, so generated or retrieved results require verification.
- Semantic retrieval quality depends on chunking, embedding quality, document extraction, and the selected retrieval settings.
- OCR may introduce extraction errors, especially in scanned documents and complex tables.
- Extracting text from a diagram does not necessarily preserve the meaning of its visual connections.
- Citation correctness depends on the accuracy of extracted metadata and the validation method.
- Issue detection and revision comparison may identify candidate differences that require engineering judgment.
- Live Gemini performance depends on API availability, model availability, configuration, and network conditions.
- Offline retrieval evaluation does not, by itself, establish the factual correctness of live language-model responses.
- The prototype has not been demonstrated to satisfy formal automotive functional-safety certification or production deployment requirements.

16. Security, Reliability, and Responsible Use

Automotive architecture documents may contain confidential technical information. A practical deployment should apply access controls, protect API credentials, and follow the organization's approved data-handling policies.

API keys must not be embedded in source code, committed to a repository, or exposed in application logs. Where a cloud language model is used, the organization should evaluate whether document content is permitted to be transmitted to the selected service.

The system should distinguish between retrieved evidence and generated interpretation, provide source references, and communicate uncertainty when sufficient evidence is unavailable.

Potential architecture inconsistencies must be reviewed by qualified engineers before corrective action is taken. The application must not automatically approve designs, alter authoritative documents, or replace required engineering verification and safety-governance procedures.

17. Future Scope

The project can be extended in the following directions:

1. **Larger document collections:** Evaluate the system using multiple real-world HLD documents with appropriate authorization and confidentiality controls.
2. **Improved diagram understanding:** Introduce document-layout analysis and multimodal models for interpreting architecture diagrams and connector relationships.
3. **Structured architecture graphs:** Represent components, ports, interfaces, and dependencies in a graph to support explicit relationship queries.
4. **Advanced inconsistency detection:** Develop rule-based and model-assisted checks for mismatched interface definitions, missing connections, and conflicting component descriptions.
5. **Enhanced revision analysis:** Identify additions, removals, and modifications across versions and connect each change to potentially affected components.
6. **Comprehensive evaluation:** Add larger test sets, expert-reviewed ground truth, more retrieval metrics, and separate evaluation of retrieval quality and generated-answer quality.
7. **Improved abstention:** Strengthen behavior for unanswerable questions and cases where retrieved passages do not provide adequate evidence.
8. **Enterprise integration:** Introduce approved identity management, access control, audit logging, and deployment infrastructure where required.
9. **Performance optimization:** Investigate caching, indexing improvements, and retrieval configurations for larger document collections.

10. **Engineering workflow integration:** Explore integration with architecture repositories, requirements management systems, and testing workflows.

Each extension should be evaluated independently before being used for high-impact engineering decisions.

18. Conclusion

The AUTOSAR HLD Document Analysis Assistant demonstrates an approach to improving access to automotive software architecture documentation through semantic retrieval and evidence-grounded AI.

By combining PDF processing, context-aware chunking, all-MiniLM-L6-v2 embeddings, ChromaDB, Gemini-based answer generation, and citation presentation, the application supports natural language exploration of architecture information. Its additional modules provide a foundation for architecture exploration, potential inconsistency analysis, revision comparison, and export.

The reported evaluation achieved 100% Recall@4, citation validity, citation coverage, and abstention accuracy, together with an 89.1% factual match rate, 1.403-second average query latency, and 0.60-second document ingestion time on the synthetic benchmark. These results are promising for the tested prototype, but they do not establish universal accuracy or production readiness.

The project's central contribution is a workflow that combines AI-assisted analysis with inspectable source evidence and human review. Further validation on representative industrial documents, improved diagram understanding, and more extensive evaluation will be necessary before the system can be relied upon in safety-critical engineering workflows.

19. References

1. AUTOSAR. *AUTOSAR Classic Platform: Architecture and Technical Documentation.* Official AUTOSAR website: https://www.autosar.org/
2. Lewis, P., et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *Advances in Neural Information Processing Systems (NeurIPS).*
3. Reimers, N., and Gurevych, I. (2019). “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” *Proceedings of EMNLP-IJCNLP.*

4. Chroma. *Chroma Documentation.* https://docs.trychroma.com/
5. Streamlit. *Streamlit Documentation.* https://docs.streamlit.io/
6. Google AI for Developers. *Gemini API Documentation.* https://ai.google.dev/gemini-api/docs
7. Hugging Face. *Sentence Transformers: all-MiniLM-L6-v2 Model Documentation.* https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
8. Automotive Engineering AI. *Standard Project Case Study Document: AUTOSAR HLD Document Analysis Assistant.* Version 1.0, September 2026. Internal project reference document.

Saurabh Mali

